Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab# 1)

Apply modern simulation tools ModelSim and Logisim to implement basic processor components

Student Name: **Ahmed Javed Qureshi** Roll No: **24i-6554** Section: **CE-B**

Date performed: **19 August, 2026**

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:
In this task I wrote codes of all gates e.g, AND , OR , NOT etc. on MODELSIM. I wrote the codes in Gate level modeling and then I compiled the codes one by one and then I simulate the codes and then I run the codes and got the required Signal Diagrams of all gates that I pasted in output.
Code:

AND:

```
module Logicgates(a,b,c);
input a;
input b;
output c;
and a1(c,b,a);
endmodule
module testbench();
reg x;
reg y;
wire z;
Logicgates uut(x,y,z);
initial
begin
x = 0; y = 0 ;
#50
x = 0 ; y = 1;
#50
x = 1 ; y = 0;
#50
x = 1 ; y = 1;
end
endmodule
```

OR:

```
Ln#
 1  module Logicgates(a,b,c);
 2  input a;
 3  input b;
 4  output c;
 5  or a2(c,b,a);
 6  endmodule
 7  module testbench();
 8  reg x;
 9  reg y;
10  wire z;
11  Logicgates uut(x,y,z);
12  initial
13  begin
14  x = 0; y = 0 ;
15  #50
16  x = 0 ; y = 1;
17  #50
18  x = 1 ; y = 0;
19  #50
20  x = 1 ; y = 1;
21  end
22  endmodule
23
```

NOT:

```
module logicgate(a,b);

input a;
output b;
not a1(b,a);
endmodule
module testbench();
reg x;
wire y;
logicgate uut(x,y);
initial
begin

x=0;
#50
x=1;

end

endmodule
```

NAND:

```
n#
1   module Logicgates(a,b,c);
2   input a;
3   input b;
4   output c;
5   nand a1(c,b,a);
6   endmodule
7   module testbench();
8   reg x;
9   reg y;
10  wire z;
11  Logicgates uut(x,y,z);
12  initial
13  begin
14  x = 0; y = 0 ;
15  #50
16  x = 0 ; y = 1;
17  #50
18  x = 1 ; y = 0;
19  #50
20  x = 1 ; y = 1;
21  end
22  endmodule
23
```

NOR:

C:/intelFPGA/18.1/DataFlow.v (/testbench) - Default *

```
Ln#
 1  module Logicgates(a,b,c);
 2  input a;
 3  input b;
 4  output c;
 5  nor a7(c,b,a);
 6  endmodule
 7  module testbench();
 8  reg x;
 9  reg y;
10  wire z;
11  Logicgates uut(x,y,z);
12  initial
13  begin
14  x = 0; y = 0 ;
15  #50
16  x = 0 ; y = 1;
17  #50
18  x = 1 ; y = 0;
19  #50
20  x = 1 ; y = 1;
21  end
22  endmodule
23
```

XOR:

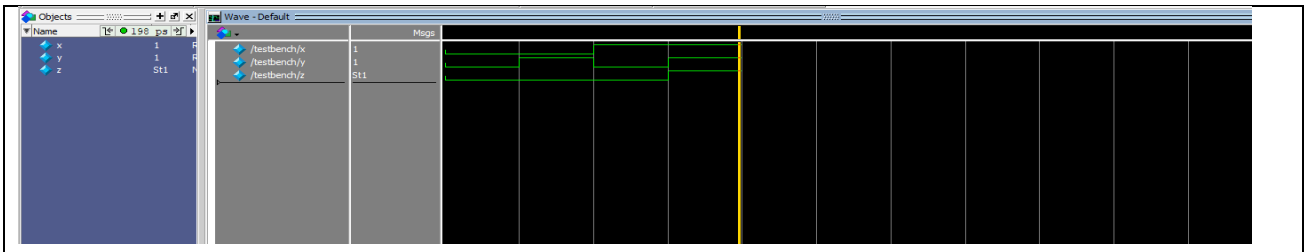
```
Ln#
 1  module Logicgates(a,b,c);
 2  input a;
 3  input b;
 4  output c;
 5  xor a7(c,b,a);
 6  endmodule
 7  module testbench();
 8  reg x;
 9  reg y;
10  wire z;
11  Logicgates uut(x,y,z);
12  initial
13  begin
14  x = 0; y = 0 ;
15  #50
16  x = 0 ; y = 1;
17  #50
18  x = 1 ; y = 0;
19  #50
20  x = 1 ; y = 1;
21  end
22  endmodule
23
```

X NOR:

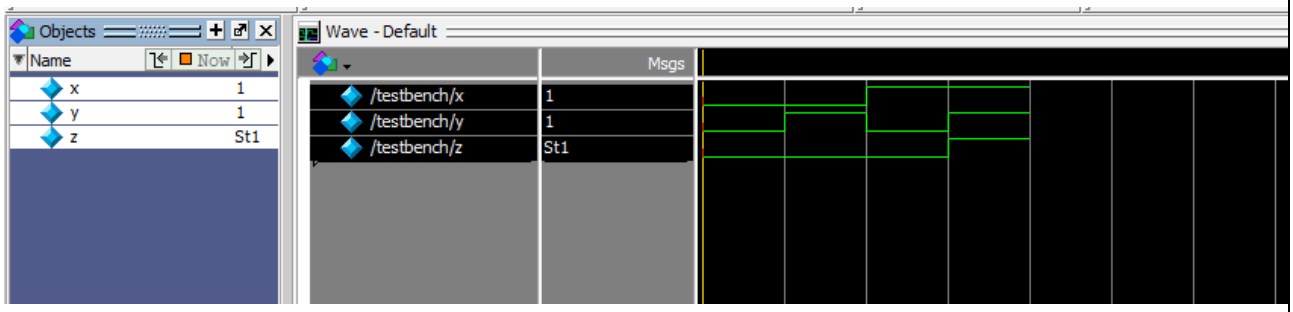
```
1  module Logicgates(a,b,c);
2  input a;
3  input b;
4  output c;
5  xnor a7(c,b,a);
6  endmodule
7  module testbench();
8  reg x;
9  reg y;
10 wire z;
11 Logicgates uut(x,y,z);
12 initial
13 begin
14 x = 0; y = 0 ;
15 #50
16 x = 0 ; y = 1;
17 #50
18 x = 1 ; y = 0;
19 #50
20 x = 1 ; y = 1;
21 end
22 endmodule
23
```

Output:

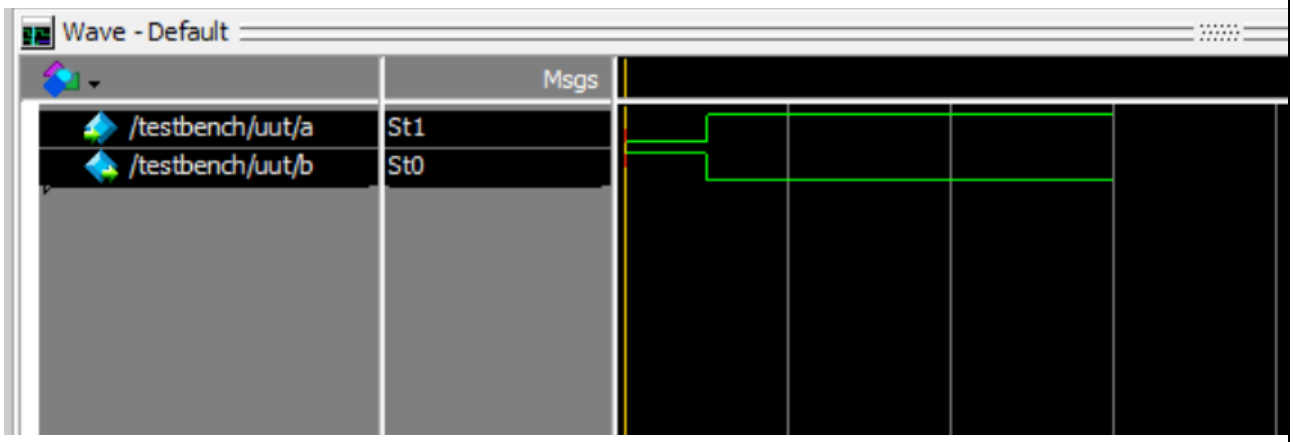
AND:



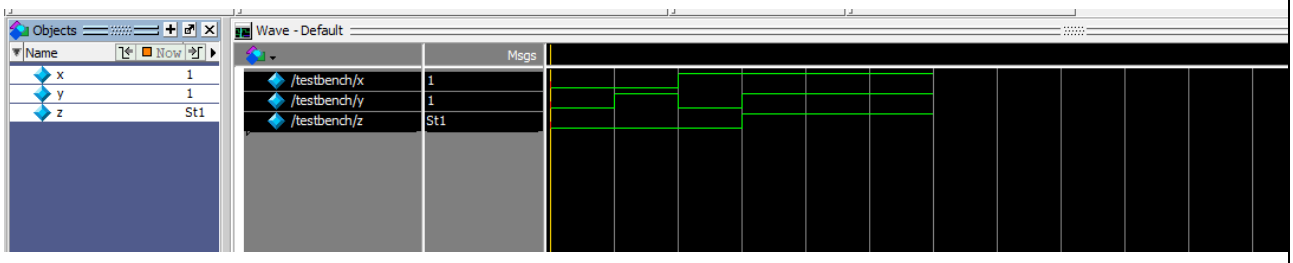
OR:


Objects
Name
Now
x
1
y
1
z
St1
Wave - Default
Msgs
/testbench/x
1
/testbench/y
1
/testbench/z
St1

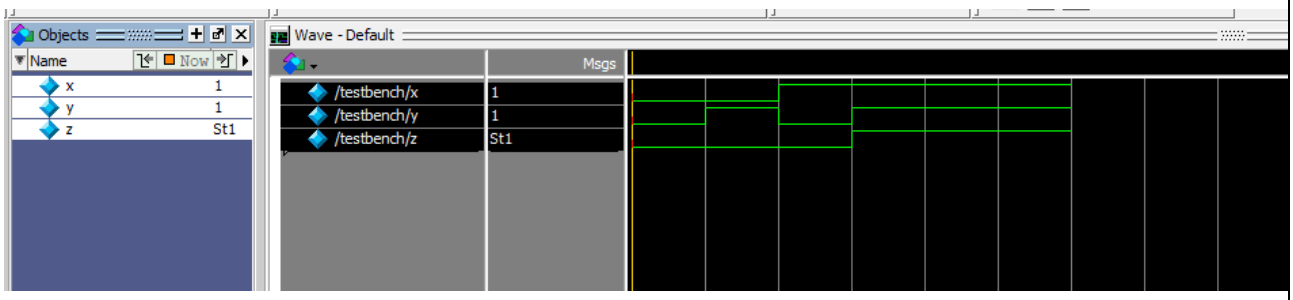
NOT:


Wave - Default
Msgs
/testbench/uut/a
St1
/testbench/uut/b
St0

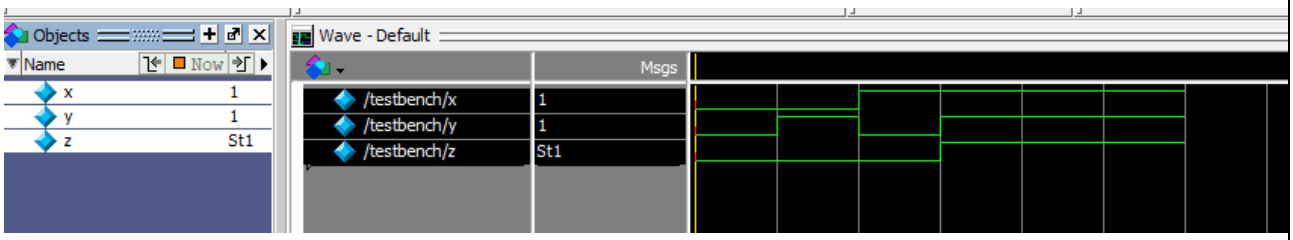
NAND:


Objects
Name
Now
x
1
y
1
z
St1
Wave - Default
Msgs
/testbench/x
1
/testbench/y
1
/testbench/z
St1

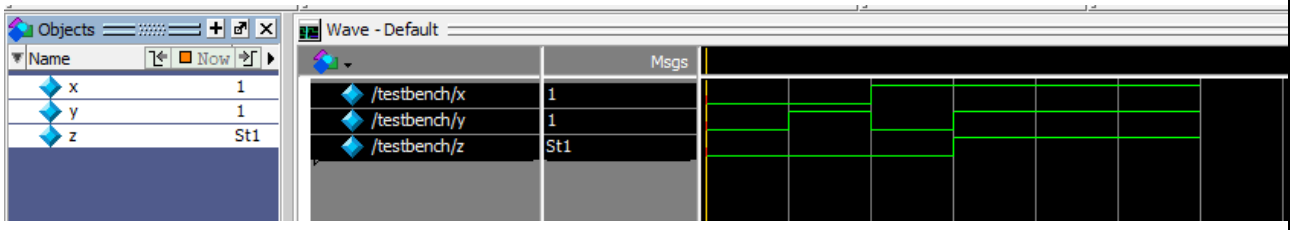
NOR:


Objects
Name
Now
x
1
y
1
z
St1
Wave - Default
Msgs
/testbench/x
1
/testbench/y
1
/testbench/z
St1

XOR:


Objects
Name
Now
x
1
y
1
z
St1
Wave - Default
Msgs
/testbench/x
1
/testbench/y
1
/testbench/z
St1

XNOR:


Objects
Name
Now
x
1
y
1
z
St1
Wave - Default
Msgs
/testbench/x
1
/testbench/y
1
/testbench/z
St1

Task 2:

Description:
In this task I wrote codes of all gates e.g, AND , OR , NOT etc. on MODELSIM. I wrote the codes in Data Flow modeling and then I compiled the codes one by one and then I simulate the codes and then I run the codes and got the required Signal Diagrams of all gates that I pasted in output.
Code:
AND GATE:

```
Ln#
 1  module Logicgates(a,b,c);
 2  input a;
 3  input b;
 4  output c;
 5  assign c = a & b;
 6  endmodule
 7  module testbench();
 8  reg x;
 9  reg y;
10  wire z;
11  Logicgates uut(x,y,z);
12  initial
13  begin
14  x = 0; y = 0;
15  #50
16  x = 0; y = 1;
17  #50
18  x = 1; y = 0;
19  #50
20  x = 1; y = 1;
21  end
22  endmodule
23
```

OR GATE:

```
1  module Logicgates(a,b,c);
2  input a;
3  input b;
4  output c;
5  assign c = a | b;
6  endmodule
7  module testbench();
8  reg x;
9  reg y;
10 wire z;
11 Logicgates uut(x,y,z);
12 initial
13 begin
14 x = 0; y = 0;
15 #50
16 x = 0; y = 1;
17 #50
18 x = 1; y = 0;
19 #50
20 x = 1; y = 1;
21 end
22 endmodule
23
```

NOT GATE:

```
Ln#
 1  module Logicgates(a,c);
 2  input a;
 3  output c;
 4  assign c = ~a;
 5  endmodule
 6  module testbench();
 7  reg x;
 8  wire z;
 9  Logicgates uut(x,z);
10
11  initial
12  begin
13  x = 0;
14  #50
15  x = 1;
16  end
17
18  endmodule
19
```

NOR GATE:

```
Ln#
 1  module Logicgates(a,b,c);
 2  input a;
 3  input b;
 4  output c;
 5  assign c = ~(a | b);
 6  endmodule
 7  module testbench();
 8  reg x;
 9  reg y;
10  wire z;
11  Logicgates uut(x,y,z);
12  initial
13  begin
14  x = 0; y = 0;
15  #50
16  x = 0; y = 1;
17  #50
18  x = 1; y = 0;
19  #50
20  x = 1; y = 1;
21  end
22  endmodule
23
```

NAND GATE:

```
module Logicgates(a,b,c);
input a;
input b;
output c;
assign c = ~(a & b);
endmodule
module testbench();
reg x;
reg y;
wire z;
Logicgates uut(x,y,z);
initial
begin
x = 0; y = 0;
#50
x = 0; y = 1;
#50
x = 1; y = 0;
#50
x = 1; y = 1;
end
endmodule
```

XOR GATE:

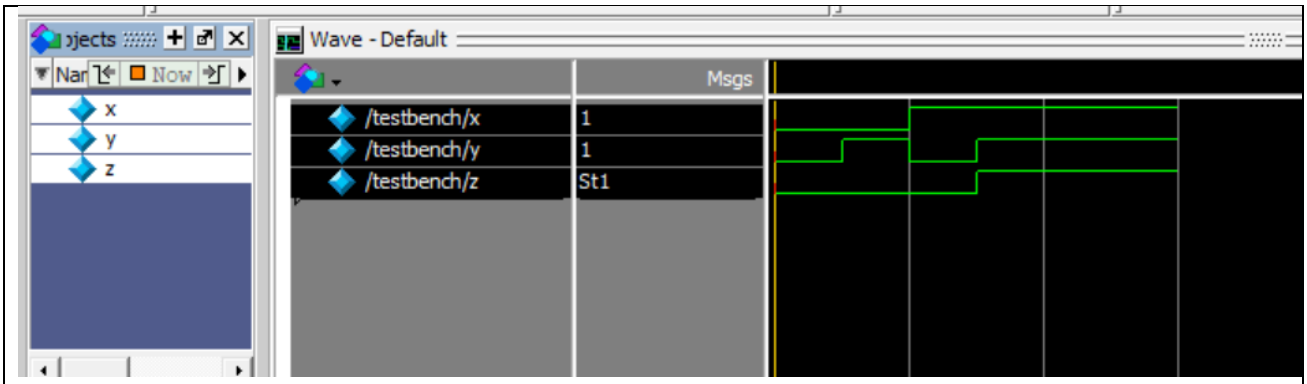
```
Ln#
 1  module Logicgates(a,b,c);
 2  input a;
 3  input b;
 4  output c;
 5  assign c = a ^ b;
 6  endmodule
 7  module testbench();
 8  reg x;
 9  reg y;
10  wire z;
11  Logicgates uut(x,y,z);
12  initial
13  begin
14  x = 0; y = 0;
15  #50
16  x = 0; y = 1;
17  #50
18  x = 1; y = 0;
19  #50
20  x = 1; y = 1;
21  end
22  endmodule
23
```

XNOR GATE:

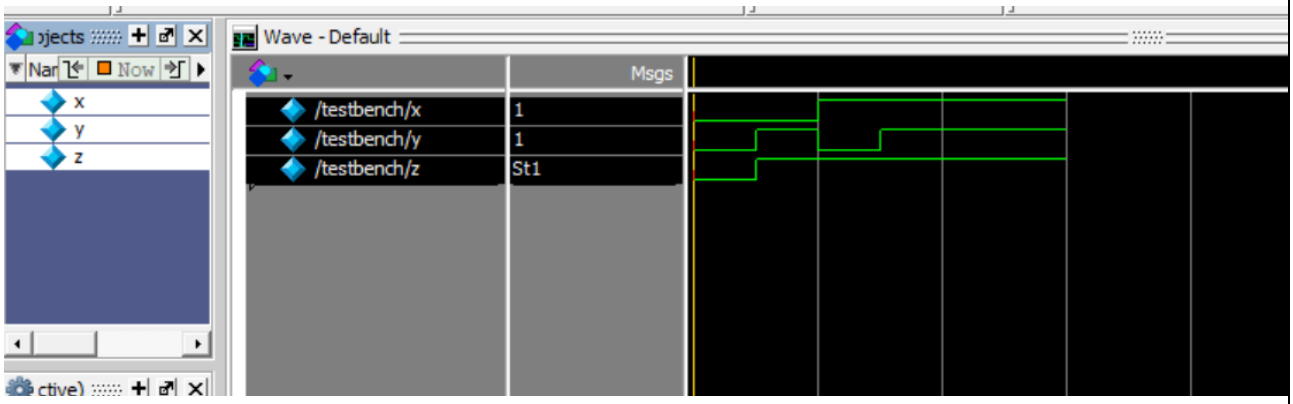
```
Ln#
 1  module Logicgates(a,b,c);
 2  input a;
 3  input b;
 4  output c;
 5  assign c = ~(a ^ b);
 6  endmodule
 7  module testbench();
 8  reg x;
 9  reg y;
10  wire z;
11  Logicgates uut(x,y,z);
12  initial
13  begin
14  x = 0; y = 0;
15  #50
16  x = 0; y = 1;
17  #50
18  x = 1; y = 0;
19  #50
20  x = 1; y = 1;
21  end
22  endmodule
23
```

Output:

AND GATE:



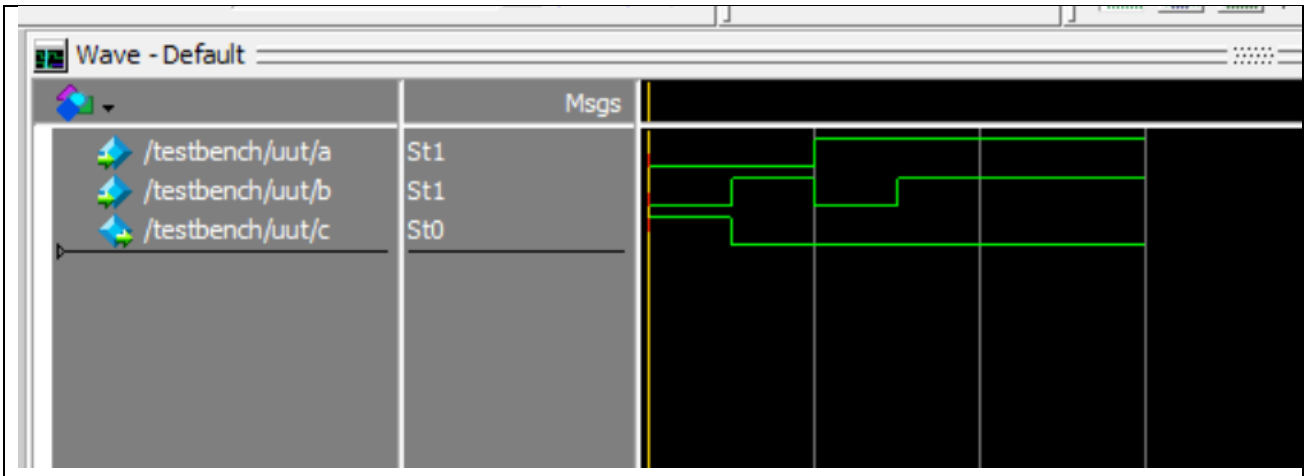
OR GATE:



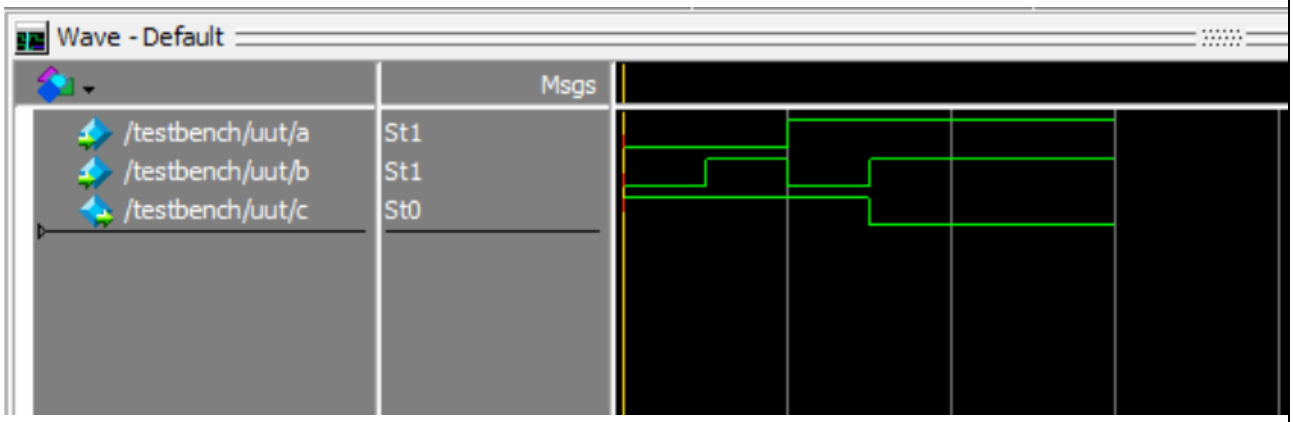
NOT GATE:



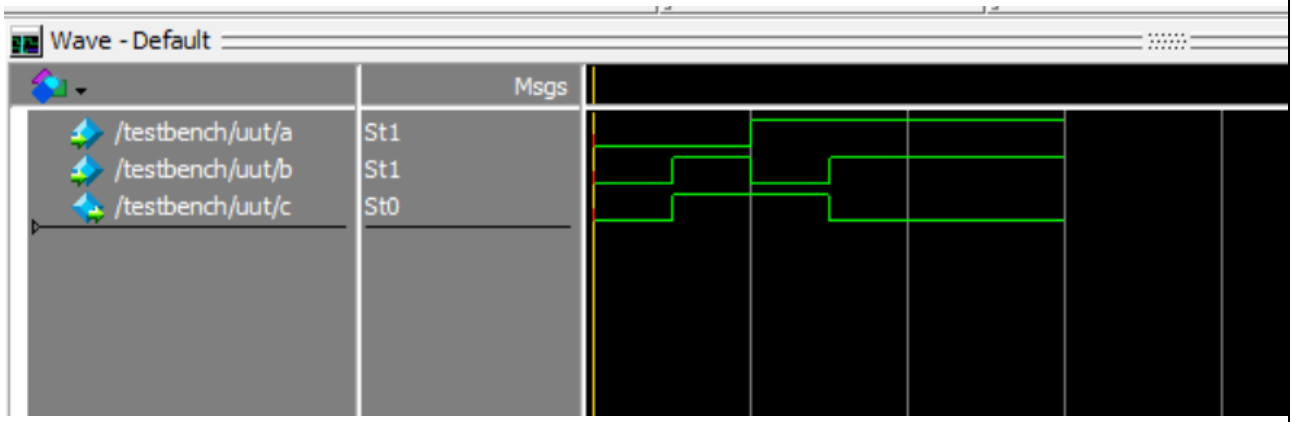
NOR GATE:



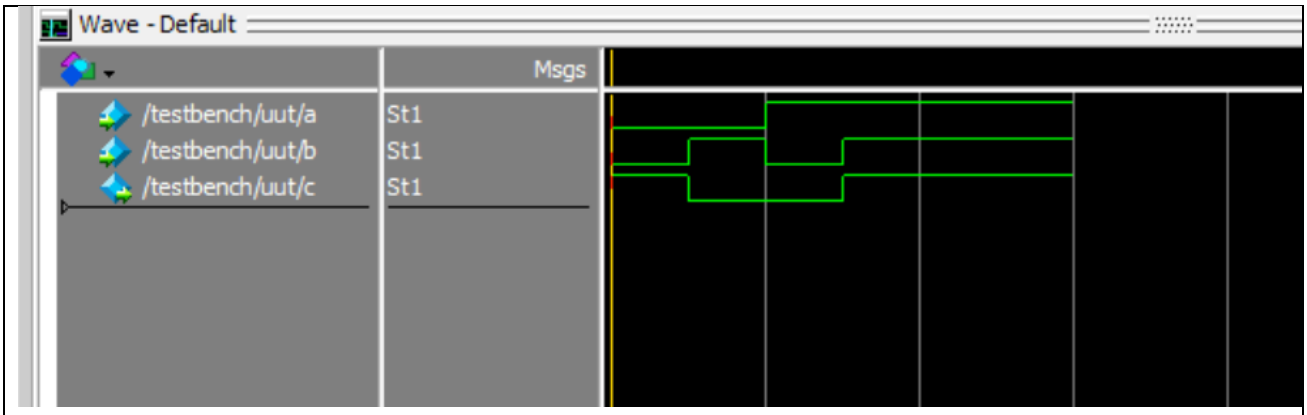
NAND GATE:



XOR GATE:



XNOR GATE:

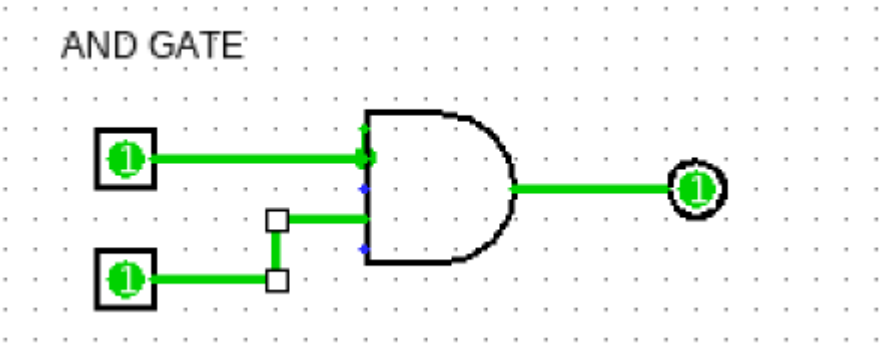


Task 3:

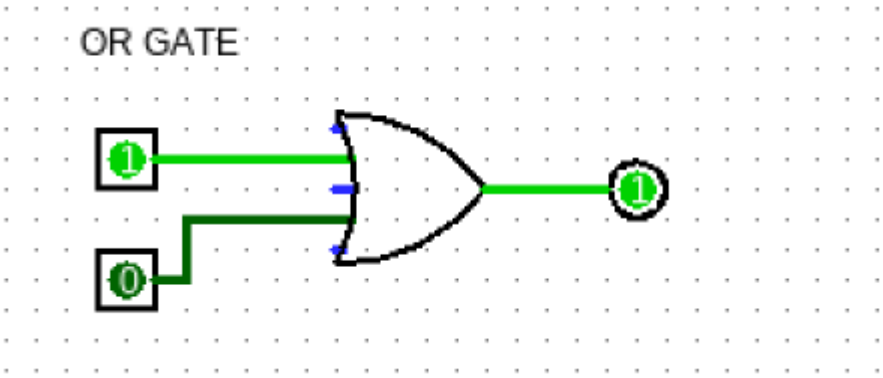
Description:
In this task I designed the circuits of all basic gates, e.g; AND , OR ,NOT etc. on logisim and then I checked the circuit by giving inputs and checking outputs. And the circuits were working completely correct .
Code:

Output:

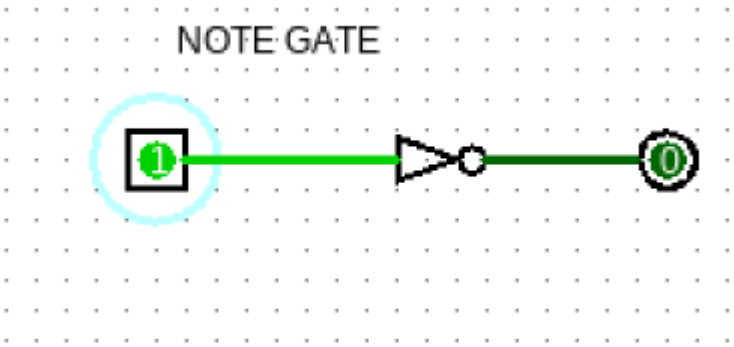
AND GATE:



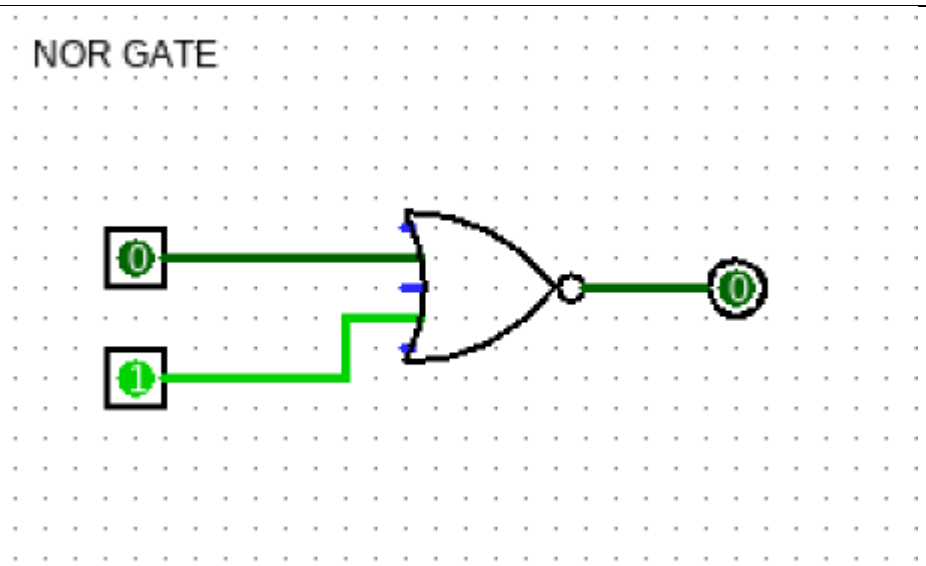
OR GATE:



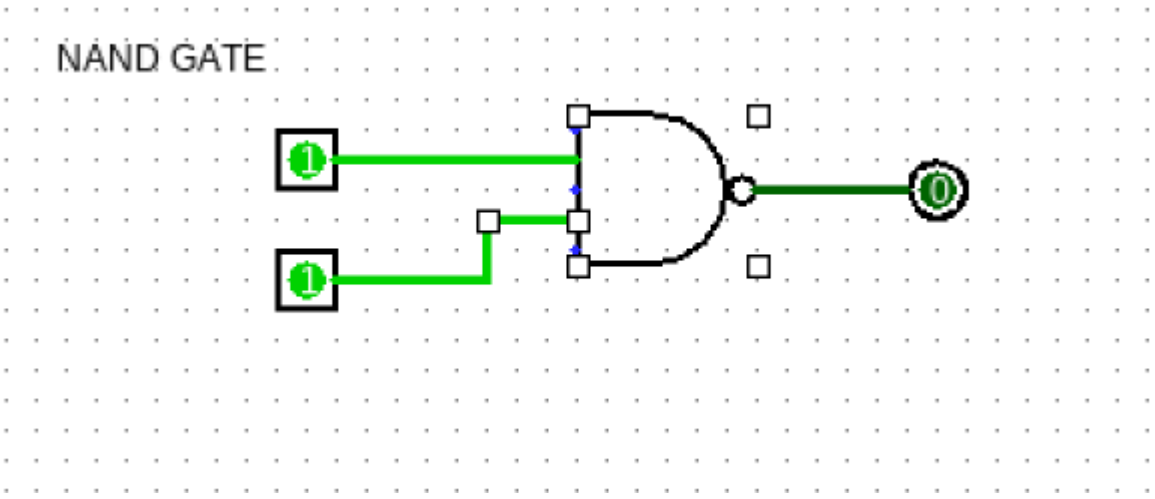
NOT GATE:



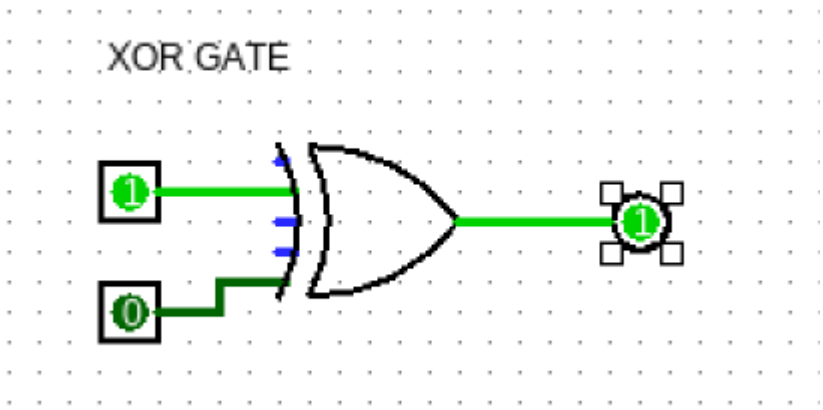
NOR GATE:



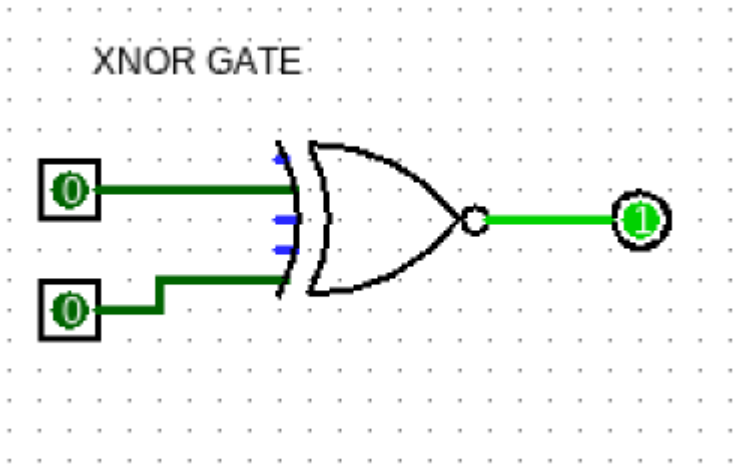
NAND GATE:



XOR GATE:



X NOR GATE:



Task 4:

Description:

In this task, I implemented the given circuit using both gate-level and dataflow modeling to make the circuit. In gate-level modeling, I used NOT, AND, and OR gates. In dataflow modeling, I used the Boolean expression of the circuit. And then , I also used a testbench to check the output for all input combinations.

Code:

GATE LEVEL:

C:/intelFPGA/18.1/LogicXor.v (/testbench/uut) - Default *

```
Ln#
 1  module logicXor(a,b,y);
 2  input a;
 3  input b;
 4  output y;
 5  wire na;
 6  wire nb;
 7  wire x;
 8  wire z;
 9  not n1(na,a);
10  not n2(nb,b);
11  and a1(x,na,b);
12  and a2(z,a,nb);
13  or o1(y,x,z);
14  endmodule
15  module testbench();
16  reg s;
17  reg t;
18  wire y;
19  logicXor uut(s,t,y);
20  initial
21  begin
22  s=0; t=0;
23  #50
```

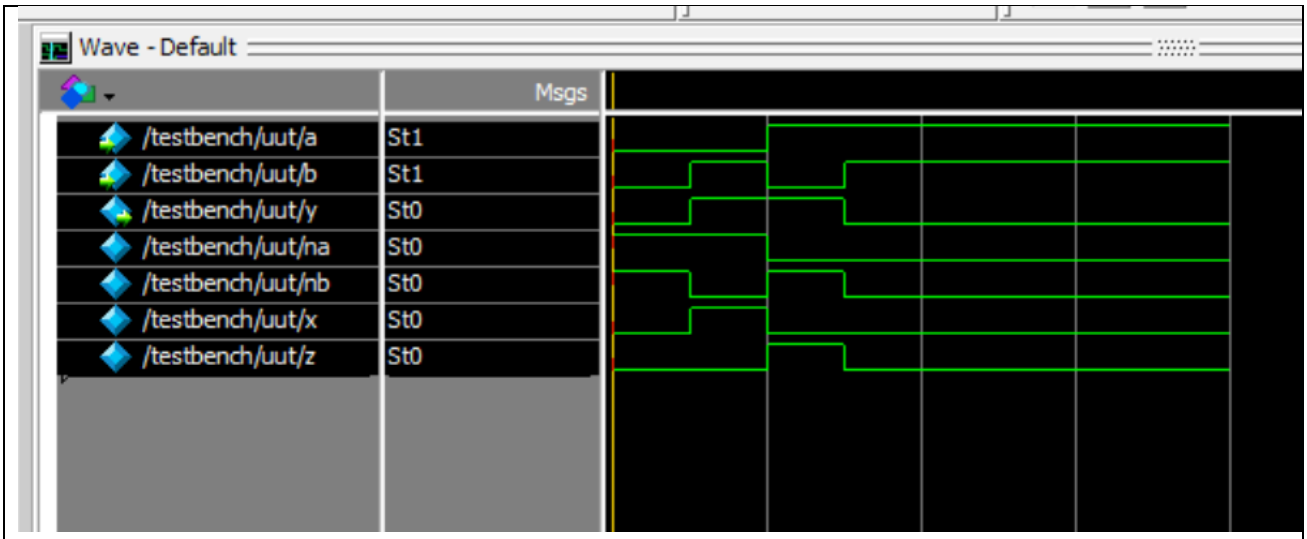
DATA FLOW:

C:/intelFPGA/18.1/LogicXor.v (/testbench/uut) - Default *

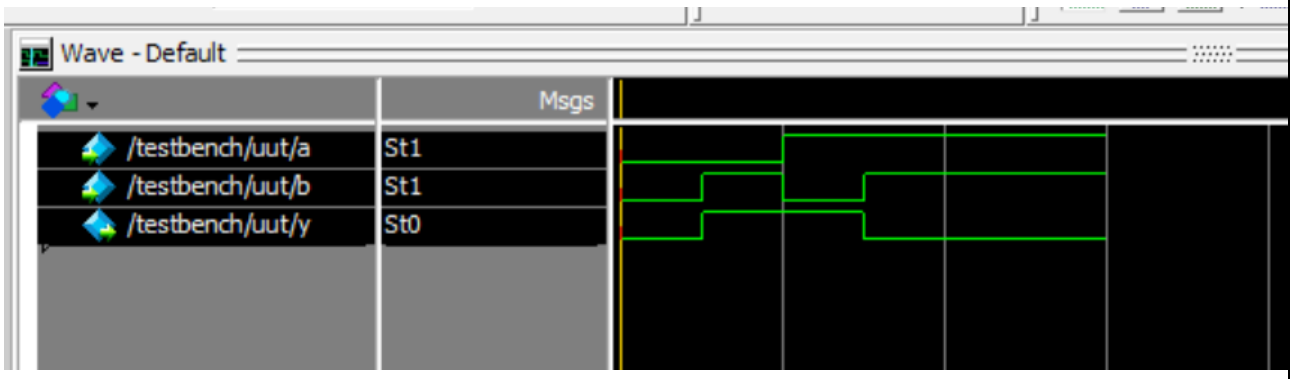
```
Ln#
 1  module logicgate(a,b,y);
 2  input a;
 3  input b;
 4  output y;
 5  assign y = (~a & b) | (a & ~b);
 6  endmodule
 7  module testbench();
 8  reg s;
 9  reg t;
10  wire y;
11  logicgate uut(s,t,y);
12  initial
13  begin
14  s=0; t=0;
15  #50
16  s=0; t=1;
17  #50
18  s=1; t=0;
19  #50
20  s=1; t=1;
21  end
22  endmodule
23
```

Output:

GATE LEVEL:



DATA FLOW:



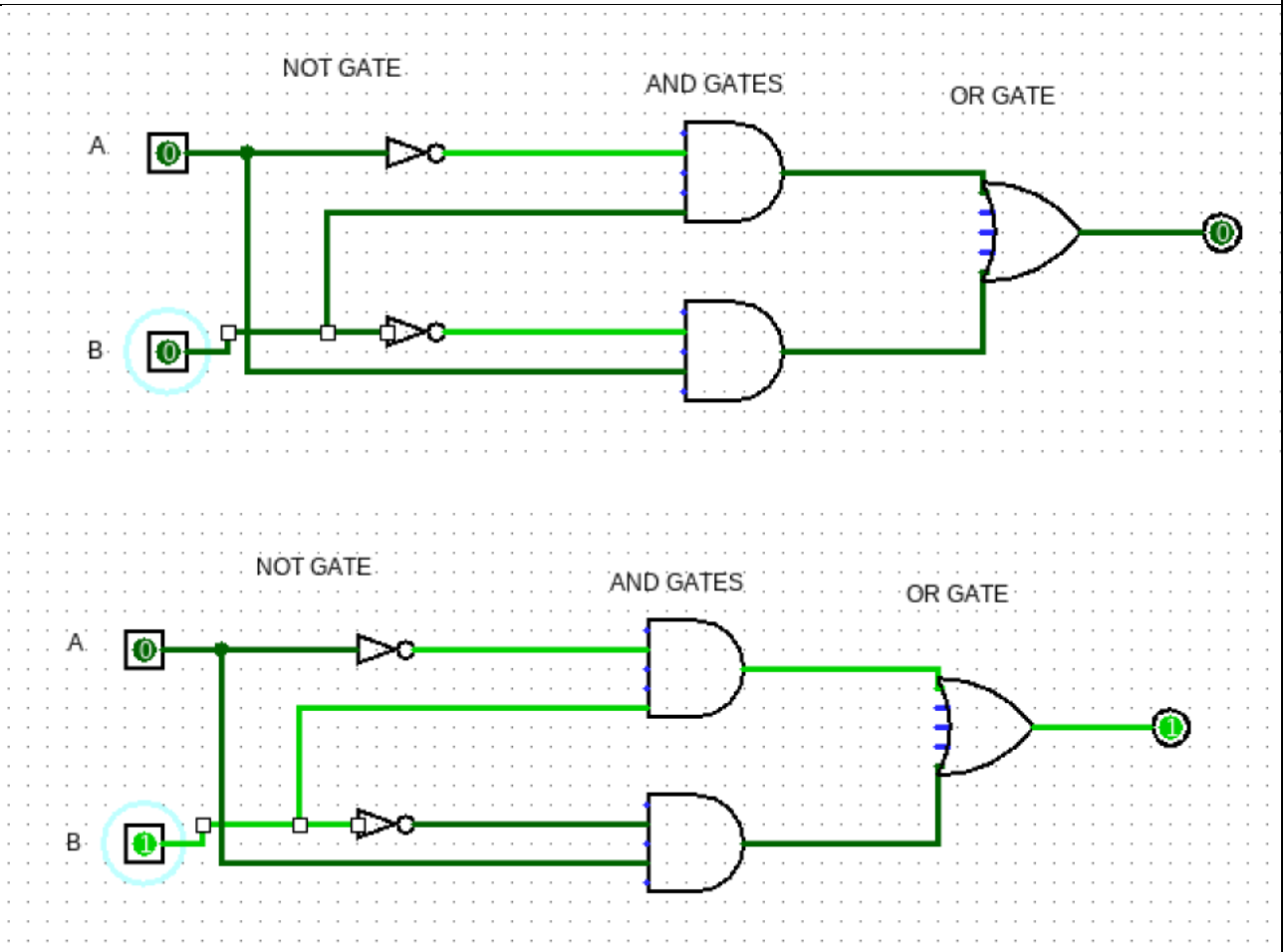
Task 5:

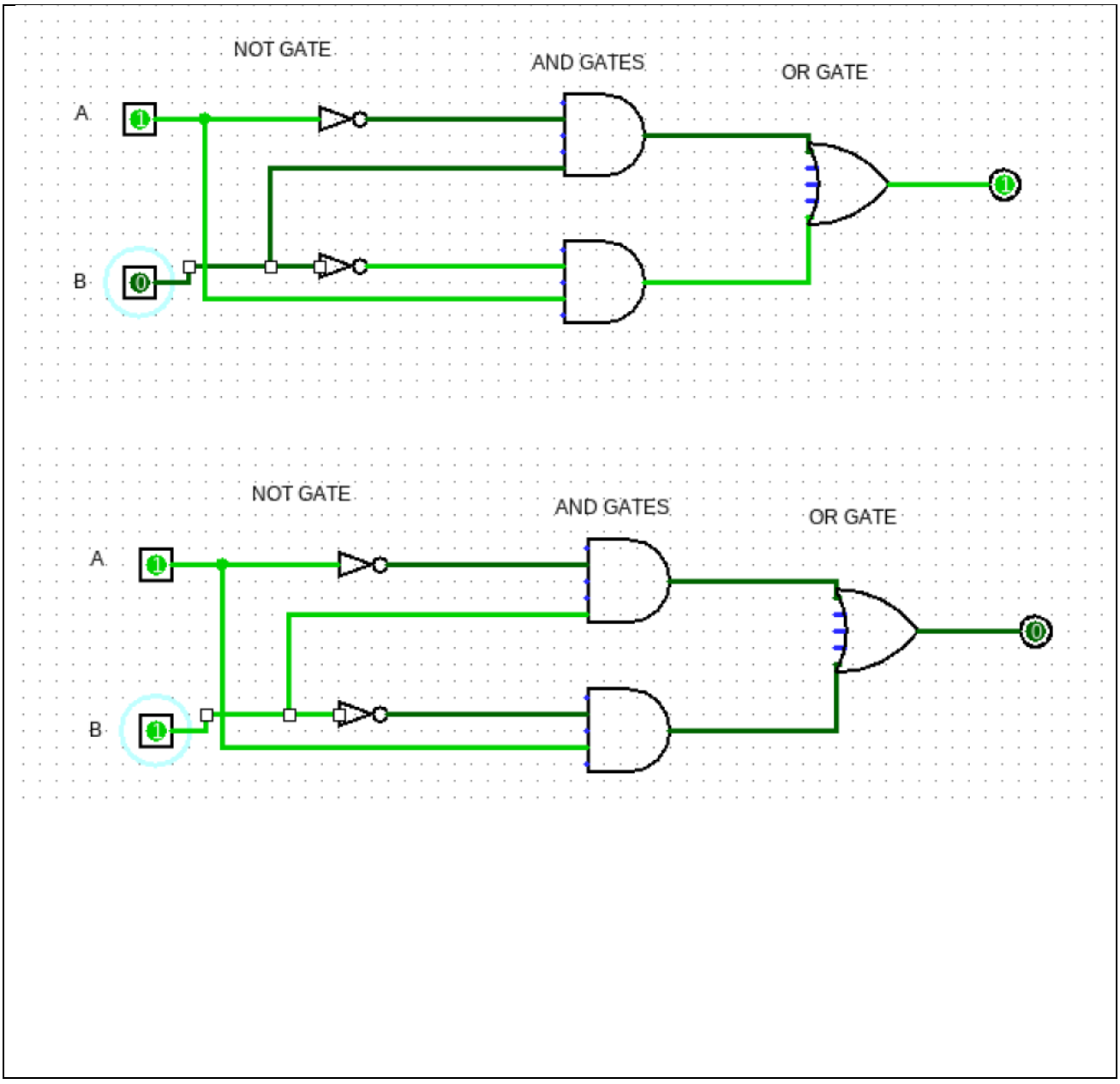
Description:
In this task I checked the working of the circuit that was given in task 4 and that diagram was of the XOR gate I then designed it on Logisim using basic gates (AND, OR , NOT). And then I tested the working of

XOR gate by giving different inputs, and the output obtained was correct. I used 2 NOT gates, 2 AND GATES and 1 OR gate .

Code:

Output:





Task 6:

Description:

In this task, I implemented the 1's complement of a 4-bit binary number using Verilog**.** The circuit takes four input bits (a, b, c, d) and produces four output bits (w, x, y, z). Each input bit is inverted, so 0 becomes 1 and 1 becomes 0. I implemented the circuit in both gate-level and dataflow modeling**.** In gate-level modeling, I used four NOT gates, while in dataflow modeling**,** assign statements were used to invert each bit. A testbench was also created to apply all 16 possible combinations of four input bits and verify that the outputs were correct.

Code:

GATE LEVEL:

```
1  module logicgate(a,b,c,d,w,x,y,z);
2  input a;
3  input b;
4  input c;
5  input d;
6  output w;
7  output x;
8  output y;
9  output z;
10 not n1(w,a);
11 not n2(x,b);
12 not n3(y,c);
13 not n4(z,d);
14 endmodule
15 module testbench();
16 reg s;
17 reg t;
18 reg u;
19 reg v;
20 wire w;
21 wire x;
22 wire y;
23 wire z;
```

```
24    logicgate uut(s,t,u,v,w,x,y,z);
25    initial
26    begin
27    s=0; t=0; u=0; v=0;
28    #50
29    s=0; t=0; u=0; v=1;
30    #50
31    s=0; t=0; u=1; v=0;
32    #50
33    s=0; t=0; u=1; v=1;
34    #50
35    s=0; t=1; u=0; v=0;
36    #50
37    s=0; t=1; u=0; v=1;
38    #50
39    s=0; t=1; u=1; v=0;
40    #50
41    s=0; t=1; u=1; v=1;
42    #50
43    s=1; t=0; u=0; v=0;
44    #50
45    s=1; t=0; u=0; v=1;
```

```
46      #50
47      s=1; t=0; u=1; v=0;
48      #50
49      s=1; t=0; u=1; v=1;
50      #50
51      s=1; t=1; u=0; v=0;
52      #50
53      s=1; t=1; u=0; v=1;
54      #50
55      s=1; t=1; u=1; v=0;
56      #50
57      s=1; t=1; u=1; v=1;
58      end
59      endmodule
60
```

DATA FLOW:

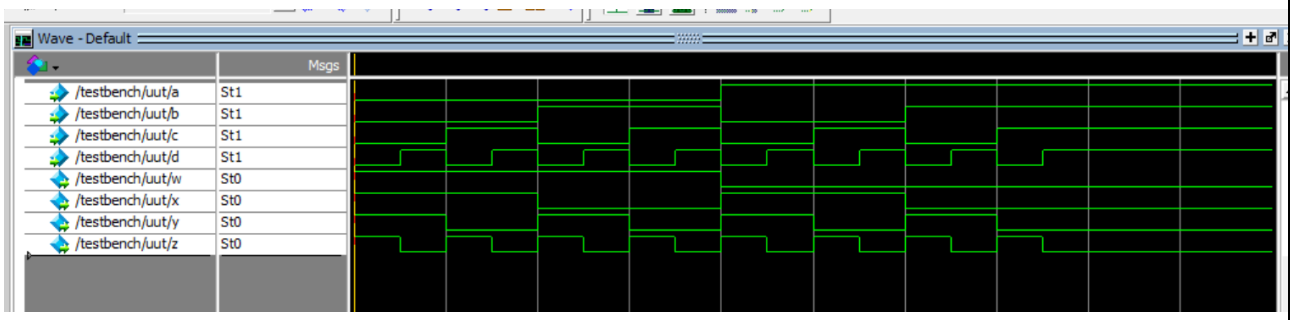
```
Ln#
 1  module logicgate(a,b,c,d,w,x,y,z);
 2  input a;
 3  input b;
 4  input c;
 5  input d;
 6  output w;
 7  output x;
 8  output y;
 9  output z;
10  assign w = ~a;
11  assign x = ~b;
12  assign y = ~c;
13  assign z = ~d;
14  endmodule
15  module testbench();
16  reg s;
17  reg t;
18  reg u;
19  reg v;
20  wire w;
21  wire x;
22  wire y;
23  wire z;
```

```
24    logicgate uut(s,t,u,v,w,x,y,z);
25    initial
26    begin
27    s=0; t=0; u=0; v=0;
28    #50
29    s=0; t=0; u=0; v=1;
30    #50
31    s=0; t=0; u=1; v=0;
32    #50
33    s=0; t=0; u=1; v=1;
34    #50
35    s=0; t=1; u=0; v=0;
36    #50
37    s=0; t=1; u=0; v=1;
38    #50
39    s=0; t=1; u=1; v=0;
40    #50
41    s=0; t=1; u=1; v=1;
42    #50
43    s=1; t=0; u=0; v=0;
```

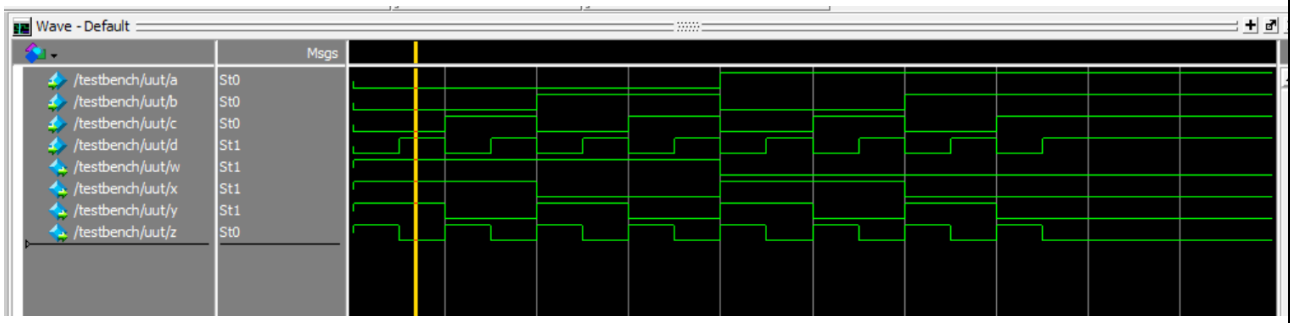
```
44      #50
45      s=1; t=0; u=0; v=1;
46      #50
47      s=1; t=0; u=1; v=0;
48      #50
49      s=1; t=0; u=1; v=1;
50      #50
51      s=1; t=1; u=0; v=0;
52      #50
53      s=1; t=1; u=0; v=1;
54      #50
55      s=1; t=1; u=1; v=0;
56      #50
57      s=1; t=1; u=1; v=1;
58    end
59    endmodule
60
```

Output:

GATE LEVEL:



DATA FLOW:


Wave - Default
Msgs
/testbench/uut/a
/testbench/uut/b
/testbench/uut/c
/testbench/uut/d
/testbench/uut/w
/testbench/uut/x
/testbench/uut/y
/testbench/uut/z
St0
St0
St0
St1
St1
St1
St1
St0